

Green Isn't Safe

Catching rogue AI agents through their accountability chain, then containing them with a human in the loop

PASS Agent registered

PASS Has an owner

PASS Account active

FAIL Owner left. Nobody is accountable.

Sai Ravula · Alex · Muyiwa

SailPoint Navigate 2026 Hack Day · MCP Server track

Every check passes. Nobody is accountable.

What the dashboard shows

PASS invoice-bot is registered and has an owner

PASS Its account is active on the source

PASS Ownership check: 0 ownerless objects

What the chain shows

FAIL Owner Denise.Hunt left the company

FAIL Holds AccountsReceivable beyond its declared purpose

FAIL Violates ENFORCED SoD policy: Money In and Out

Accountability is a chain. ISC is viewed one object at a time.



No single ISC screen shows this chain, so no one checks it.

Agents inherit the gaps humans already have

0

ownerless objects

The standard check passes

96%

**of governed objects
owned by one person**

45 of 47, and she has no
manager

9 / 9

**leavers still have
enabled accounts**

27 accounts, incl. AD with
password never expires

65%

**of privileged grants are
direct**

33 of 51, with no role or
access profile

Measured live in the demo tenant today. We dropped two claims the data didn't support.

An MCP server that walks the chain

1 Explain

"Why does invoice-bot have AccountsReceivable?"

`explain_access`

2 Expose

"Is our governance actually healthy?"

`find_accountability_gaps`

3 Detect

"Are any of our AI agents rogue?"

`detect_rogue_agents`

4 Contain

"Quarantine invoice-bot. (Human confirms.)"

`quarantine_agent`

Every answer carries evidence[] (ISC object IDs and timestamps), so an auditor can replay every claim.

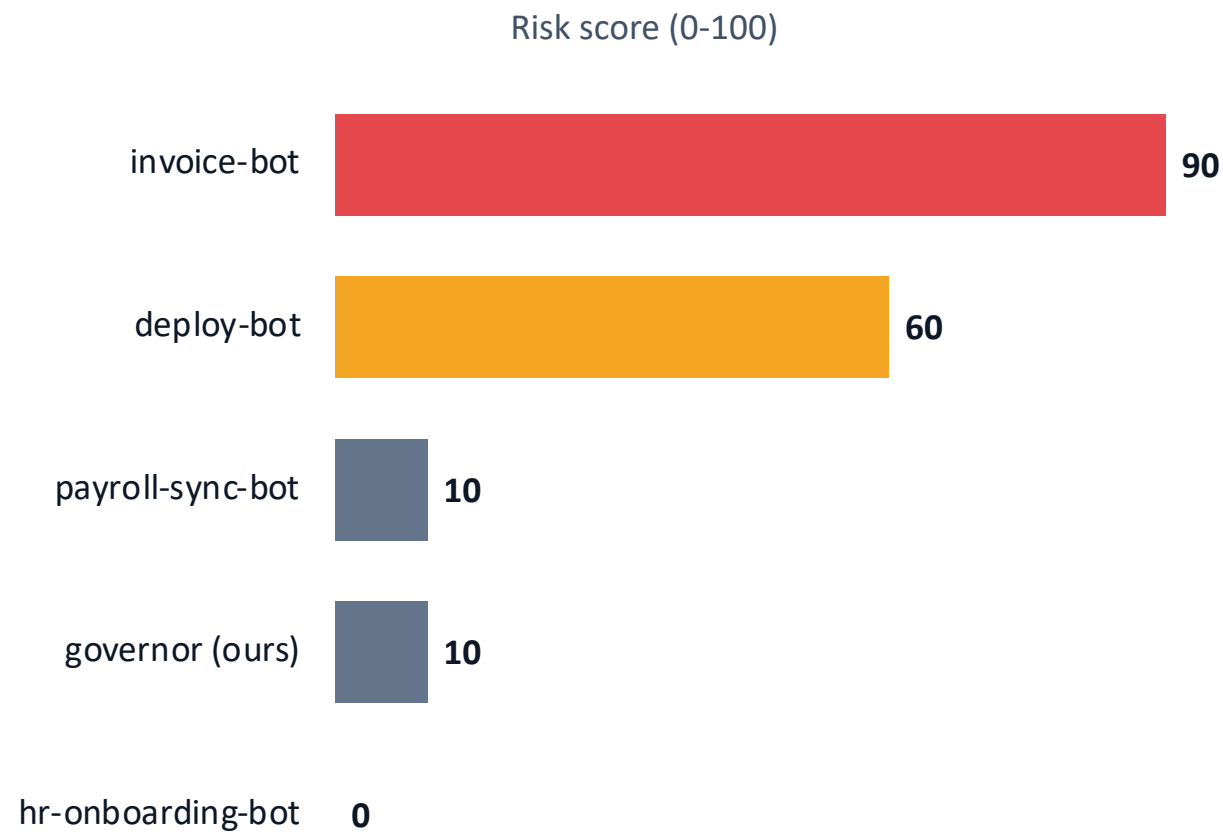
How the MCP server works



Tool	Answers	ISC APIs it calls	Mode
<code>search_identities</code>	Who is this person?	<code>/v3/search</code>	Read-only
<code>explain_access</code>	Why does X have Y?	<code>search</code> , <code>access-profiles</code> , <code>roles</code>	Read-only
<code>find_accountability_gaps</code>	Is governance healthy?	<code>roles</code> , <code>access-profiles</code> , <code>sources</code> , <code>search</code>	Read-only
<code>detect_rogue_agents</code>	Is any agent rogue?	<code>accounts</code> , <code>entitlements</code> , <code>sod-policies</code> , <code>search</code>	Read-only
<code>quarantine_agent</code>	Contain this agent	<code>tagged-objects</code> , plus the target app	Dry run, then token + human

Credentials come from macOS Keychain or Windows Credential Manager. Every ISC call is logged with a correlation ID.

One rogue agent in a fleet of five



invoice-bot

90 CRITICAL

- +40 Owner Denise.Hunt is an inactive leaver
- +30 Violates ENFORCED SoD policy (matched by entitlement ID)
- +20 AccountsReceivable is outside its declared purpose

Escalate to: Catherine.Simmons, the departed owner's manager

Agents are staged on a CSV source. Their owners, leaver status, entitlements and SoD policy are real ISC objects.

Nothing is contained until a human approves



Guardrails

4 of 5 tools read-only

quarantine_agent is marked destructive; the server refuses to act without a dry-run token

No secrets on disk

Credentials live only in macOS Keychain or Windows Credential Manager

Auditable and reversible

Every ISC call logged with a correlation ID; one command reverses containment

Your dashboards say green.

Ask the agent what they're not telling you.

One conversation

Detect, explain and contain; each tool call takes under 5 seconds

One model

The same chain logic catches rogue agents and leavers

30-day pilot

Every CRITICAL agent contained within 1 business day

github.com/sairavula/Sailpoint-Navigate26-HackDay